EN3160 – Assignment 3: Neural Networks

Index No.	230352K
Name	Kumara K.B.R
GitHub	https://github.com/buddhi-rash/EN-3160---Assesment-03

This notebook has the code, results and a short discussion for each question.

- **Q1** – Two-layer fully connected network with manual back-propagation (CIFAR-10)
- **Q2** – LeNet-5 vs a modern small CNN (MNIST)
- **Q3** – Transfer learning with ResNet18 (ants vs bees)

Question 1 — Fully Connected Network with Manual Back-propagation

Objective

Listing 1 trains a single-layer network on CIFAR-10 with MSE loss. The changes needed are:

- (a) add a hidden layer with 100 nodes and sigmoid activation,
- (b) use cross-entropy loss,
- (c) train for 10 epochs and report the training and test accuracy.

Baseline: original single-layer network

The first code cell is Listing 1 as given (20 epochs, MSE loss). I use it as the baseline.

In [ ]:
```
# Imports: PyTorch for tensors, torchvision for CIFAR-10, matplotlib for plotting
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms

# 1. Dataloading
transform = transforms.Compose(
    [
        transforms.ToTensor(),
```

```
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    ]
)
# Mini-batch size used for training and evaluation
batch_size = 50
# CIFAR-10 training set (50,000 images), normalised to [-1, 1]
trainset = torchvision.datasets.CIFAR10(root="./data", train=True, download=True, t
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=
# CIFAR-10 test set (10,000 images); not shuffled so that evaluation is determinist
testset = torchvision.datasets.CIFAR10(root="./data", train=False, download=True, t
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size, shuffle=Fa

# Class names (index -> label)
classes = (
    "plane",
    "car",
    "bird",
    "cat",
    "deer",
    "dog",
    "frog",
    "horse",
    "ship",
    "truck",
)

# 2. Define Network Parameters
Din = 3 * 32 * 32  # Input size (flattened CIFAR-10 image size)
K = 10  # Output size (number of classes in CIFAR-10)
# Standard deviation of the small random weight initialisation
std = 1e-5

# Initialize weights and biases
w = (torch.randn(Din, K) * std)  # One layer: directly map input to output
b = torch.zeros(K)

# Hyperparameters
iterations = 20
lr = 2e-6  # Learning rate
lr_decay = 0.9  # Learning rate decay
reg = 0  # Regularization
loss_history = []

# 3. Training Loop
# Each iteration of the outer loop is one epoch (a full pass over the training set)
for t in range(iterations):
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # Get inputs and labels
        inputs, labels = data
        Ntr = inputs.shape[0]  # Batch size
        x_train = inputs.view(Ntr, -1)  # Flatten input to (Ntr, Din)
        y_train_onehot = (nn.functional.one_hot(labels, K).float())  # Convert labe

        # Forward pass
        y_pred = x_train.mm(w) + b  # Output layer activation
```

```
        # Loss calculation (Mean Squared Error with regularization)
        loss = (1 / Ntr) * torch.sum((y_pred - y_train_onehot) ** 2) + reg * torch.
        loss_history.append(loss.item())
        running_loss += loss.item()

        # Backpropagation
        # Gradient of the MSE loss with respect to the predictions
        dy_pred = (2.0 / Ntr) * (y_pred - y_train_onehot)
        # Chain rule: dL/dw = x^T . dL/dy_pred (+ regularisation); dL/db sums over
        dw = x_train.t().mm(dy_pred) + reg * w
        db = dy_pred.sum(dim=0)

        # Parameter update
        # Manual SGD step, applied in place (no autograd is used)
        w -= lr * dw
        b -= lr * db

    # Print loss for every epoch
    if t % 1 == 0:
        print(
            f"Epoch {t + 1}/{iterations}, Loss: {running_loss / len(trainloader)}"
        )

    # Learning rate decay
    lr *= lr_decay

# 4. Plotting the Loss History
plt.plot(loss_history)
plt.title("Loss History")
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.show()

# 5. Calculate Accuracy on Training Set
correct_train = 0
total_train = 0
# Gradient tracking is disabled because the weights are only evaluated here
with torch.no_grad():
    for data in trainloader:
        inputs, labels = data
        Ntr = inputs.shape[0]
        x_train = inputs.view(Ntr, -1)
        y_train_onehot = nn.functional.one_hot(labels, K).float()
        # Forward pass
        y_train_pred = x_train.mm(w) + b
        # Predicted class = output unit with the highest score
        predicted_train = torch.argmax(y_train_pred, dim=1)
        total_train += labels.size(0)
        correct_train += (predicted_train == labels).sum().item()

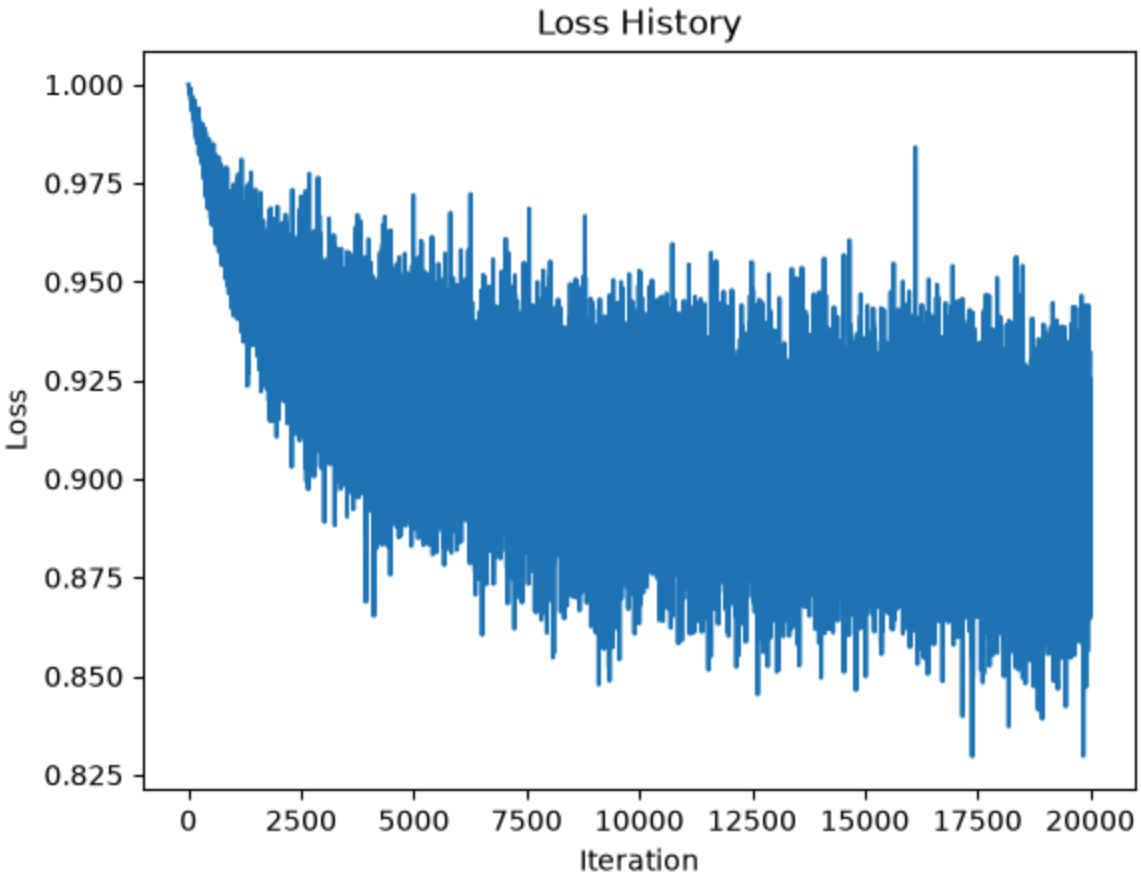
# Accuracy = percentage of correctly classified training images
train_acc = 100 * correct_train / total_train
print(f"Training accuracy: {train_acc:.2f}%")

# 6. Calculate Accuracy on Test Set
```

```
correct_test = 0
total_test = 0
with torch.no_grad():
    for data in testloader:
        inputs, labels = data
        Nte = inputs.shape[0]
        x_test = inputs.view(Nte, -1)
        y_test_onehot = nn.functional.one_hot(labels, K).float()
        # Forward pass
        y_test_pred = x_test.mm(w) + b
        predicted_test = torch.argmax(y_test_pred, dim=1)
        total_test += labels.size(0)
        correct_test += (predicted_test == labels).sum().item()

test_acc = 100 * correct_test / total_test
print(f"Test accuracy: {test_acc:.2f}%")
```

```
Epoch 1/20, Loss: 0.9769332077503204
Epoch 2/20, Loss: 0.9498110665678978
Epoch 3/20, Loss: 0.936068968653679
Epoch 4/20, Loss: 0.9275178614258767
Epoch 5/20, Loss: 0.9215836952328682
Epoch 6/20, Loss: 0.9171839657425881
Epoch 7/20, Loss: 0.9137710046768188
Epoch 8/20, Loss: 0.9110442957282067
Epoch 9/20, Loss: 0.9088167726993561
Epoch 10/20, Loss: 0.9069643502831459
Epoch 11/20, Loss: 0.9054052955508232
Epoch 12/20, Loss: 0.9040782381892204
Epoch 13/20, Loss: 0.9029399736523628
Epoch 14/20, Loss: 0.901957014977932
Epoch 15/20, Loss: 0.9011032971739769
Epoch 16/20, Loss: 0.9003584219813346
Epoch 17/20, Loss: 0.8997060278058052
Epoch 18/20, Loss: 0.8991328237056733
Epoch 19/20, Loss: 0.898627524793148
Epoch 20/20, Loss: 0.8981810814738274
```




```
Training accuracy: 32.22%
Test accuracy: 32.42%
```

Baseline result (single layer, MSE, 20 epochs)

	Accuracy
Training	32.22 %
Test	32.42 %

The loss goes down slowly (0.977 → 0.898) and the accuracy stays low. A single linear layer can only learn linear boundaries, and since the train and test accuracies are almost the same, the model is under-fitting.

Modified network (parts a and b)

Architecture: 3072 → 100 (sigmoid) → 10 (softmax)

Forward pass

$$Z_1 = XW_1 + b_1, \quad A_1 = \sigma(Z_1), \quad Z_2 = A_1W_2 + b_2, \quad P = \text{softmax}(Z_2)$$

Loss (cross-entropy, Y = one-hot labels, N = batch size)

$$L = -\frac{1}{N}\sum_{i=1}^{N} \log P_{i,y_i}$$

Back-propagation (done by hand, no autograd)

- $\frac{\partial L}{\partial Z_2} = \frac{P - Y}{N}$
- $\frac{\partial L}{\partial W_2} = A_1^\top \frac{\partial L}{\partial Z_2}$, $\frac{\partial L}{\partial b_2} = \sum_i \frac{\partial L}{\partial Z_2}$
- $\frac{\partial L}{\partial Z_1} = \left(\frac{\partial L}{\partial Z_2} W_2^\top\right) \odot A_1 \odot (1 - A_1)$
- $\frac{\partial L}{\partial W_1} = X^\top \frac{\partial L}{\partial Z_1}$, $\frac{\partial L}{\partial b_1} = \sum_i \frac{\partial L}{\partial Z_1}$

Hyperparameters

- Hidden nodes: 100, epochs: 10, batch size: 50
- Learning rate: 8×10^{-2}, multiplied by 0.9 every epoch. This is much larger than the baseline (2×10^{-6}) because the cross-entropy gradients are small and get even smaller through the sigmoid.
- Weights initialised from N(0, 10^{-5}), biases 0, no regularisation.

In [12]:
```
# Modified fully connected network with 100 unit hidden layer

# Define the sigmoid activation function
# Sigmoid activation: squashes each hidden pre-activation into (0, 1)
def sigmoid(x):
    return torch.sigmoid(x)

# 2. Define Network Parameters
Din = 3 * 32 * 32  # Input size (flattened CIFAR-10 image size)
H = 100  # Hidden layer size
K = 10  # Output size (number of classes in CIFAR-10)
std = 1e-5

# Initialize modified weights and biases
w1 = (torch.randn(Din, H) * std)  # first layer weights ( Din - H)
b1 = torch.zeros(H)
w2 = (torch.randn(H, K) * std)  # second layer weights
b2 = torch.zeros(K)

# Hyperparameters
iterations = 10
# Larger step size than the single-layer baseline (hidden-layer gradients are small
lr = 8e-2  # Learning rate
lr_decay = 0.9  # Learning rate decay
reg = 0  # Regularization
loss_history = []

# 3. Training Loop
for t in range(iterations):
    running_loss = 0.0
```

```
    for i, data in enumerate(trainloader, 0):
        # Get inputs and labels
        inputs, labels = data
        Ntr = inputs.shape[0]  # Batch size
        x_train = inputs.view(Ntr, -1)  # Flatten input to (Ntr, Din)
        y_train_onehot = (nn.functional.one_hot(labels, K).float())  # Convert labe

        # Forward pass
        # Modified using the new hidden layer
        a_1 = x_train.mm(w1) + b1  # First layer activation
        a_1 = sigmoid(a_1)  # Apply sigmoid activation
        y_pred = a_1.mm(w2) + b2  # Output layer activation

        # Loss calculation (Cross-entropy loss) calculated using PyTorch's built-in
        loss = nn.CrossEntropyLoss()(y_pred, labels) + reg * (torch.sum(w1 ** 2) +
        loss_history.append(loss.item())
        running_loss += loss.item()

        # Backpropagation
        # Modified Backpropagation for the two-layer network
        # Softmax converts the output scores (logits) into class probabilities
        y_pred_probs = torch.softmax(y_pred, dim=1)
        # For softmax + cross-entropy, the gradient w.r.t. the logits is (p - y) /
        dy_pred =  (y_pred_probs - y_train_onehot) / Ntr

        # Gradients of the output (second) layer
        dw2 = a_1.t().mm(dy_pred) + reg * w2
        db2 = dy_pred.sum(dim=0)

        # Backpropagate to the hidden layer
        da_1 = dy_pred.mm(w2.t())
        dz_1 = da_1 * (a_1 * (1 - a_1))  # Derivative of sigmoid

        # Compute gradients for the first layer
        dw1 = x_train.t().mm(dz_1) + reg * w1
        db1 = dz_1.sum(dim=0)

        # Parameter update
        # SGD update of both layers' weights and biases
        w1 -= lr * dw1
        b1 -= lr * db1
        w2 -= lr * dw2
        b2 -= lr * db2

    # Print loss for every epoch
    if t % 1 == 0:
        print(
            f"Epoch {t + 1}/{iterations}, Loss: {running_loss / len(trainloader)}"
        )

    # Learning rate decay
    lr *= lr_decay

# 4. Plotting the Loss History
plt.plot(loss_history)
plt.title("Loss History")
```

```
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.show()

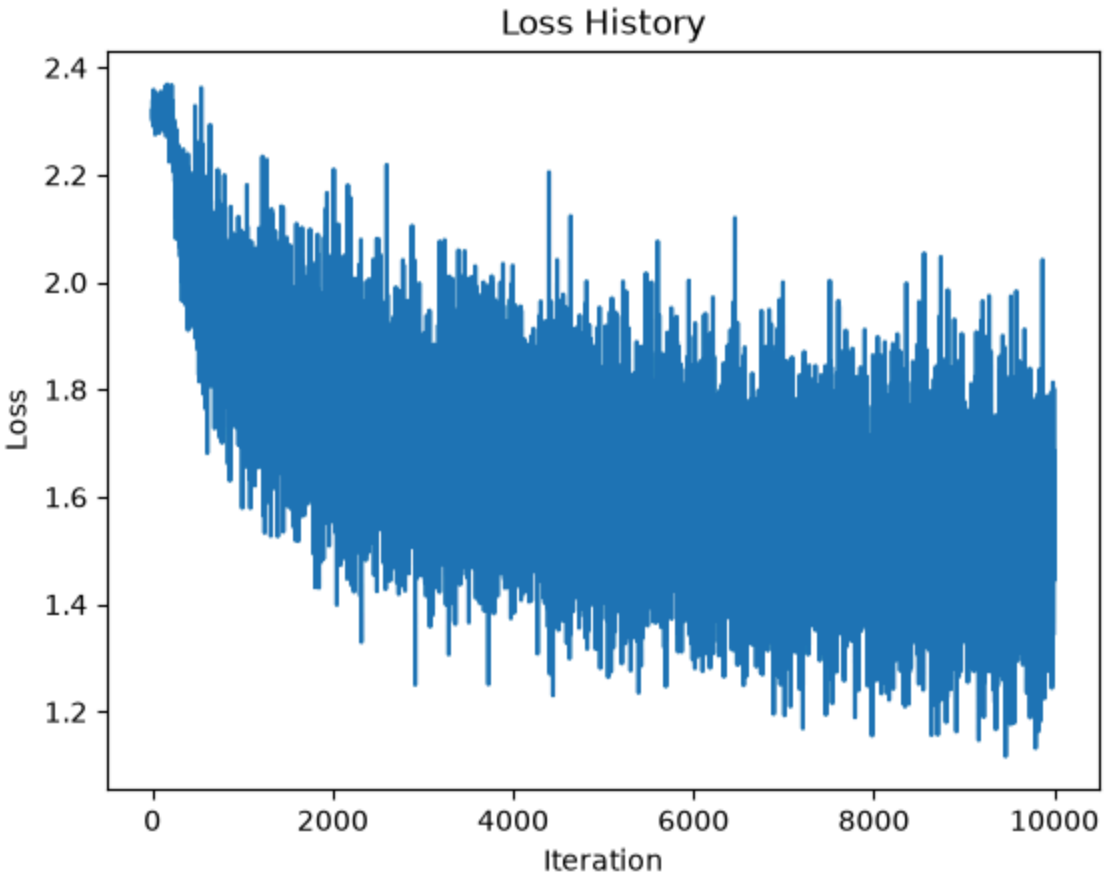
# 5. Calculate Accuracy on Training Set
correct_train = 0
total_train = 0
# Gradient tracking is disabled because the weights are only evaluated here
with torch.no_grad():
    for data in trainloader:
        inputs, labels = data
        Ntr = inputs.shape[0]
        x_train = inputs.view(Ntr, -1)
        y_train_onehot = nn.functional.one_hot(labels, K).float()
        # Forward pass
        a_1 = x_train.mm(w1) + b1
        a_1 = sigmoid(a_1)
        y_train_pred = a_1.mm(w2) + b2
        predicted_train = torch.argmax(y_train_pred, dim=1)
        total_train += labels.size(0)
        correct_train += (predicted_train == labels).sum().item()

train_acc = 100 * correct_train / total_train
print(f"Training accuracy: {train_acc:.2f}%")

# 6. Calculate Accuracy on Test Set
correct_test = 0
total_test = 0
with torch.no_grad():
    for data in testloader:
        inputs, labels = data
        Nte = inputs.shape[0]
        x_test = inputs.view(Nte, -1)
        y_test_onehot = nn.functional.one_hot(labels, K).float()
        # Forward pass
        a_1 = x_test.mm(w1) + b1
        a_1 = sigmoid(a_1)
        y_test_pred = a_1.mm(w2) + b2
        predicted_test = torch.argmax(y_test_pred, dim=1)
        total_test += labels.size(0)
        correct_test += (predicted_test == labels).sum().item()

test_acc = 100 * correct_test / total_test
print(f"Test accuracy: {test_acc:.2f}%")
```

```
Epoch 1/10, Loss: 2.0711891919374468
Epoch 2/10, Loss: 1.819261487364769
Epoch 3/10, Loss: 1.7291698914766311
Epoch 4/10, Loss: 1.6786141978502274
Epoch 5/10, Loss: 1.641053901195526
Epoch 6/10, Loss: 1.6091494407653808
Epoch 7/10, Loss: 1.58250422167778
Epoch 8/10, Loss: 1.5585657439231873
Epoch 9/10, Loss: 1.5366098914146424
Epoch 10/10, Loss: 1.5181932312250137
```




```
Training accuracy: 48.11%
Test accuracy: 45.82%
```

Results and Discussion (part c)

Model	Loss	Epochs	Train acc.	Test acc.
Baseline: single layer	MSE	20	32.22 %	32.42 %
Modified: 100 sigmoid hidden nodes	**Cross-entropy**	**10**	**48.11 %**	**45.82 %**

Observations

- The loss goes down steadily, from 2.07 to 1.52. It starts near ln 10 ≈ 2.30, which is what random guessing over 10 classes gives.
- The test accuracy improved from 32.42 % to 45.82 % in fewer epochs. This is because of the hidden layer (non-linearity) and the cross-entropy loss. I changed both at once, so I can't say how much each one helped.
- Train accuracy is only about 2 points higher than test accuracy, so there is little over-fitting. The loss is still falling at epoch 10, so more epochs would help.
- A fully connected network ignores the spatial structure of images, so its accuracy on CIFAR-10 is limited. CNNs are better for this, as seen in Q2 and Q3.

Question 2 — LeNet-5 and a Modern Small CNN on MNIST

Objective

- (a) Build LeNet-5 for MNIST and report the train and test accuracy after 10 epochs.
- (b) Build a modern small CNN (3 conv layers with batch norm and dropout) and train it the same way.
- (c) Plot the loss and accuracy per epoch and get the confusion matrix on the test set for both models.

Training setup

Both models use the same settings so the comparison is fair.

- Data: MNIST (60 000 train, 10 000 test), padded from 28×28 to 32×32 and normalised
- Optimiser: Adam, learning rate 10^{-3}, multiplied by 0.9 after each epoch
- Loss: cross-entropy with an L2 penalty of 10^{-4} on the weights
- Batch size 50, 10 epochs
- Train and test metrics are calculated in `eval()` mode after every epoch

Architectures

LeNet-5 (about 61.7 k parameters): conv 5×5 (6 filters) → avg-pool → conv 5×5 (16 filters) → avg-pool → FC 120 → FC 84 → FC 10, with tanh activations.

Modern CNN (about 105.9 k parameters): three blocks of conv 3×3 (8, 24, 48 filters) + batch norm + ReLU + max-pool, then FC 120 → dropout (0.3) → FC 10.

In [2]:
```
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
# sklearn utilities for computing and plotting the confusion matrix
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
# Creating LeNet 5 using pytorch


# Scaled tanh used in the original LeNet-5 paper: f(x) = 1.7159 * tanh(2x / 3)
def lenet_tanh(x):
    return 1.7159 * torch.tanh((2.0 / 3.0) * x)

# LeNet 5
```

```
# LeNet-5: two conv + average-pool stages followed by three fully connected layers
class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5)  # Input channels: 1 (grayscale
        self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)  # Average pooling layer
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)  # Input channels: 6, Output c
        self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)  # Second average poolin
        self.fc1 = nn.Linear(400, 120)  # Fully connected layer
        self.fc2 = nn.Linear(120, 84)  # Fully connected layer
        self.fc3 = nn.Linear(84, 10)  # Output layer for 10 classes

    # Forward pass: 1x32x32 -> [conv, pool] x 2 -> flatten (400) -> 120 -> 84 -> 10
    def forward(self, x):
        x = lenet_tanh(self.conv1(x))  # Apply tanh activation after first convolut
        x = self.pool1(x)  # Apply average pooling
        x = lenet_tanh(self.conv2(x))  # Apply tanh activation after second convolu
        x = self.pool2(x)  # Apply average pooling
        x = torch.flatten(x, 1)  # Flatten the tensor for fully connected layers
        x = lenet_tanh(self.fc1(x))  # Apply tanh activation after first fully conn
        x = lenet_tanh(self.fc2(x))  # Apply tanh activation after second fully con
        x = self.fc3(x)  # Output layer (no activation here as we'll use CrossEntro
        return x

# Modern small CNN: 3 x [conv -> batch norm -> ReLU -> max-pool], then a dropout-re
class CNN_classifier(nn.Module):
    def __init__(self, p_drop=0.3):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, kernel_size=3,padding=1)  # Input channels: 1
        self.norm1 = nn.BatchNorm2d(8)  # Batch normalization layer for 8 channels
        self.conv2 = nn.Conv2d(8, 24, kernel_size=3,padding=1)  # Input channels: 8
        self.norm2 = nn.BatchNorm2d(24)  # Batch normalization layer for 24 channel
        self.conv3 = nn.Conv2d(24, 48, kernel_size=3,padding=1)  # Input channels:
        self.norm3 = nn.BatchNorm2d(48)  # Batch normalization layer for 48 channel
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)  # max pooling layer
        self.act = nn.ReLU()  # ReLU activation function
        self.dropout = nn.Dropout(p_drop)
        self.fc1 = nn.Linear(4*4*48, 120)  # Fully connected layer
        self.fc2 = nn.Linear(120, 10)  # Output layer for 10 classes

    # Forward pass: 1x32x32 -> 3 conv blocks (spatial size 32 -> 16 -> 8 -> 4) -> f
    def forward(self, x):
        x = self.pool(self.act(self.norm1(self.conv1(x))))  # Apply batch normaliza
        x = self.pool(self.act(self.norm2(self.conv2(x))))  # Apply batch normaliza
        x = self.pool(self.act(self.norm3(self.conv3(x))))  # Apply batch normaliza
        x = torch.flatten(x, 1)  # Flatten the tensor for fully connected layers
        x = self.act(self.fc1(x))  # Apply ReLU activation after first fully connec
        x = self.dropout(x)
        x = self.fc2(x)  # Output layer (no activation here as we'll use CrossEntro
        return x

# Training the models
# Training routine shared by both models; logs per-epoch loss and accuracy on the t
def train(model, epochs, lr, lr_decay, reg):

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
    model.to(device)  # Move the model to the appropriate device (GPU or CPU)

    # Adam optimiser (the learning rate is decayed manually after every epoch)
    optimizer = optim.Adam(model.parameters(), lr=lr)
    loss_criterian = nn.CrossEntropyLoss()

    # Lists that store the per-epoch metrics for plotting
    train_losses = []
    test_losses = []
    train_accuracies = []
    test_accuracies = []

    for epoch in range(epochs):
        model.train()  # Set the model to training mode

        for i, data in enumerate(trainloader_MNIST, 0):
            inputs, labels = data
            inputs, labels = inputs.to(device), labels.to(device)

            # Forward pass
            outputs = model(inputs)

            # L2 penalty on weight tensors only (dim > 1); biases and batch-norm pa
            l2 = sum((p ** 2).sum() for p in model.parameters() if p.dim() > 1)
            loss = loss_criterian(outputs, labels) + reg * l2

            # Backpropagation
            # Standard update: clear old gradients, back-propagate, take an optimis
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        # Learning rate decay
        for param_group in optimizer.param_groups:
            param_group['lr'] *= lr_decay

        # End-of-epoch evaluation on the full training and test sets (eval mode: dr
        epoch_train_loss, epoch_train_accuracy = evaluate_model(model, trainloader_
        epoch_test_loss, epoch_test_accuracy = evaluate_model(model, testloader_MNI

        train_losses.append(epoch_train_loss)
        test_losses.append(epoch_test_loss)
        train_accuracies.append(epoch_train_accuracy)
        test_accuracies.append(epoch_test_accuracy)

        print(
            f"Epoch {epoch + 1}/{epochs} | "
            f"Train Loss: {epoch_train_loss:.4f} | "
            f"Train Accuracy: {epoch_train_accuracy:.2f}% | "
            f"Test Loss: {epoch_test_loss:.4f} | "
            f"Test Accuracy: {epoch_test_accuracy:.2f}%")

    return model, train_losses, test_losses, train_accuracies, test_accuracies

# Computes the average loss and accuracy on a data loader; optionally returns predi
def evaluate_model(model, dataloader,  loss_criterian, device, return_preds_labels=
```

```
    # validation Step after each epoch
    model.eval()  # Set the model to evaluation mode
    correct = 0
    loss = 0.0
    all_preds, all_labels = [], []
    with torch.no_grad():
        for data in dataloader:
            inputs, labels = data
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss += loss_criterian(outputs, labels).item() * inputs.size(0)

            predicted = torch.argmax(outputs, dim=1)
            correct += (predicted == labels).sum().item()
            if return_preds_labels:
                all_preds.append(predicted.cpu())
                all_labels.append(labels.cpu())

        loss = loss / len(dataloader.dataset)
        accuracy = 100 * correct / len(dataloader.dataset)

    if return_preds_labels:
        return loss, accuracy, torch.cat(all_preds).numpy(), torch.cat(all_labels).
    return loss, accuracy

# Plots the training/test loss and accuracy curves side by side
def plot_graphs(tr_loss,tr_acc, te_loss, te_acc):

    epochs = range(1, len(tr_loss) + 1)

    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.plot(epochs, tr_loss, label='Train Loss')
    plt.plot(epochs, te_loss, label='Test Loss')
    plt.title('Loss over Epochs')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(epochs, tr_acc, label='Train Accuracy')
    plt.plot(epochs, te_acc, label='Test Accuracy')
    plt.title('Accuracy over Epochs')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy (%)')
    plt.legend()

    plt.tight_layout()
    plt.show()

# Main experiment: both models are trained with identical settings
if __name__ == "__main__":
```

```
# Hyperparameters shared by both models
epochs = 10
lr = 0.001
lr_decay = 0.9
reg = 0.0001
batch_size = 50

criterion = nn.CrossEntropyLoss()
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

transform_MNIST = transforms.Compose([
    transforms.Pad(2),  # 28x28 -> 32x32
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,)) # Mnist dataset mean and std for
    ])

trainset_MNIST = torchvision.datasets.MNIST(root="./data_MNIST", train=True, do
trainloader_MNIST = torch.utils.data.DataLoader(trainset_MNIST, batch_size=batc
testset_MNIST = torchvision.datasets.MNIST(root="./data_MNIST", train=False, do
testloader_MNIST = torch.utils.data.DataLoader(testset_MNIST, batch_size=batch_

# Train and plot graphs for LeNet-5
# ---- Model 1: LeNet-5 ----
model = LeNet5()

print(f"Training LeNet-5 on MNIST dataset for {epochs} epochs with lr={lr}, lr_
trained_model, train_losses, test_losses, train_accuracies, test_accuracies = t
print(f"Final: Train Acc = {train_accuracies[-1]:.2f}% | Test Acc = {test_accur
plot_graphs(train_losses, train_accuracies, test_losses, test_accuracies)

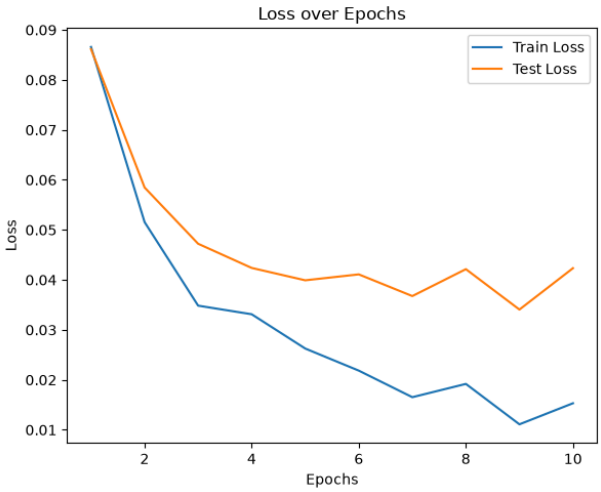

# Confusion matrix on the test set (rows: true digit, columns: predicted digit)
loss, acc, preds, labels = evaluate_model(trained_model, testloader_MNIST, crit
cm = confusion_matrix(labels, preds)
ConfusionMatrixDisplay(cm, display_labels=range(10)).plot(cmap="Blues", values_
plt.title(f"LeNet-5 Confusion Matrix (Test Acc: {acc:.2f}%)")
plt.show()

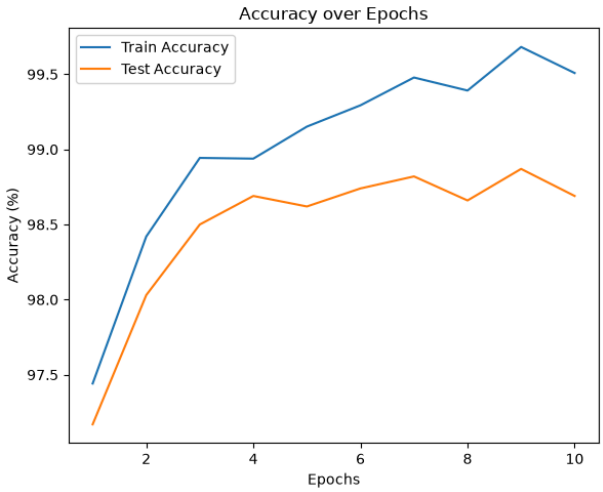
# Train and plot graphs for custom designed CNN_classifier
# ---- Model 2: modern CNN ----
model = CNN_classifier()

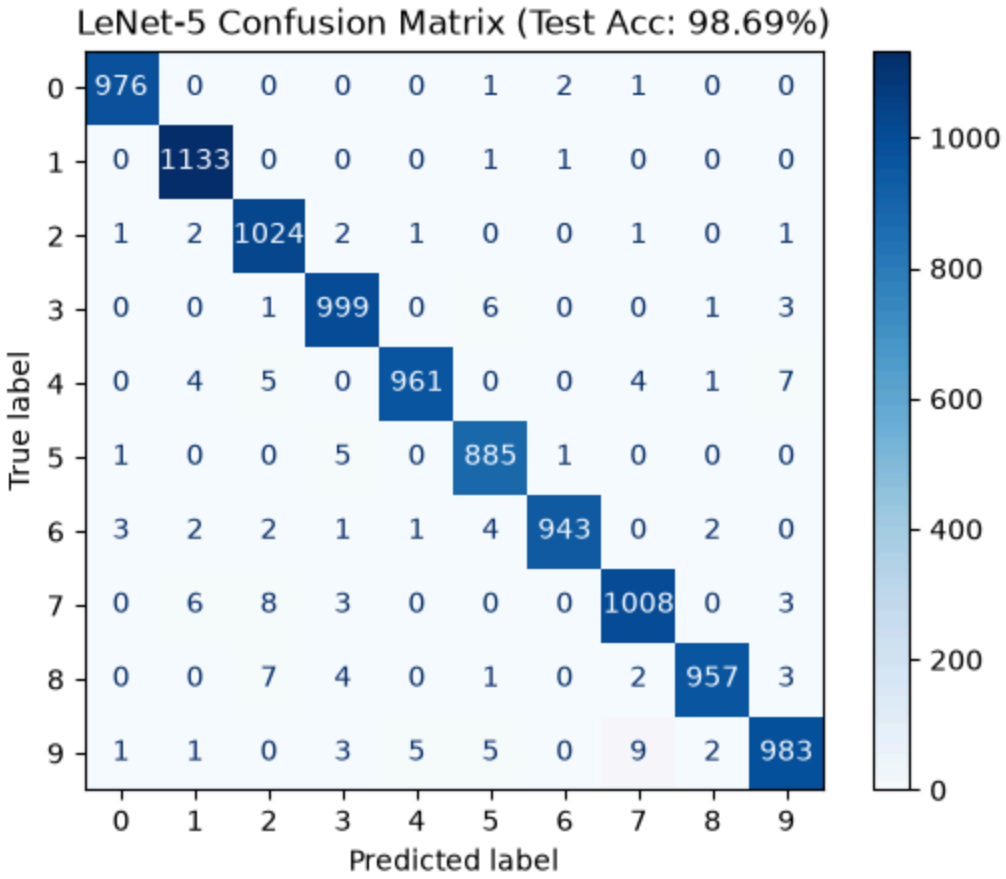
print(f"Training CNN_classifier on MNIST dataset for {epochs} epochs with lr={l
trained_model, train_losses, test_losses, train_accuracies, test_accuracies = t
print(f"Final: Train Acc = {train_accuracies[-1]:.2f}% | Test Acc = {test_accur
plot_graphs(train_losses, train_accuracies, test_losses, test_accuracies)


loss, acc, preds, labels = evaluate_model(trained_model, testloader_MNIST, crit
cm = confusion_matrix(labels, preds)
ConfusionMatrixDisplay(cm, display_labels=range(10)).plot(cmap="Blues", values_
plt.title(f"CNN_classifier Confusion Matrix (Test Acc: {acc:.2f}%)")
plt.show()
```

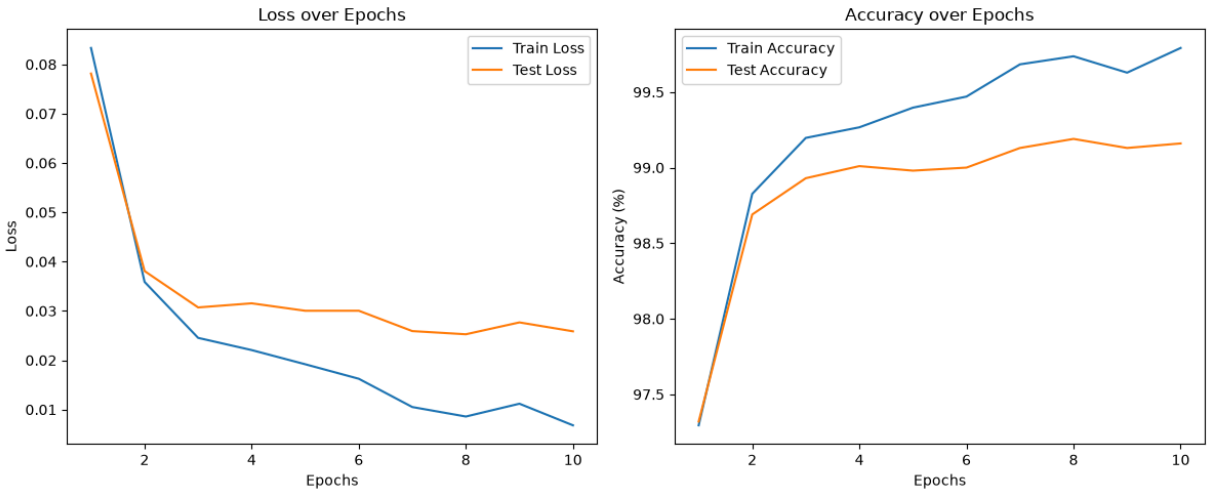
```
Training LeNet-5 on MNIST dataset for 10 epochs with lr=0.001, lr_decay=0.9, reg=0.0
001
Epoch 1/10 | Train Loss: 0.0866 | Train Accuracy: 97.44% | Test Loss: 0.0862 | Test
Accuracy: 97.17%
Epoch 2/10 | Train Loss: 0.0516 | Train Accuracy: 98.42% | Test Loss: 0.0585 | Test
Accuracy: 98.03%
Epoch 3/10 | Train Loss: 0.0348 | Train Accuracy: 98.94% | Test Loss: 0.0472 | Test
Accuracy: 98.50%
Epoch 4/10 | Train Loss: 0.0331 | Train Accuracy: 98.94% | Test Loss: 0.0424 | Test
Accuracy: 98.69%
Epoch 5/10 | Train Loss: 0.0263 | Train Accuracy: 99.15% | Test Loss: 0.0399 | Test
Accuracy: 98.62%
Epoch 6/10 | Train Loss: 0.0218 | Train Accuracy: 99.29% | Test Loss: 0.0411 | Test
Accuracy: 98.74%
Epoch 7/10 | Train Loss: 0.0165 | Train Accuracy: 99.48% | Test Loss: 0.0368 | Test
Accuracy: 98.82%
Epoch 8/10 | Train Loss: 0.0192 | Train Accuracy: 99.39% | Test Loss: 0.0421 | Test
Accuracy: 98.66%
Epoch 9/10 | Train Loss: 0.0111 | Train Accuracy: 99.68% | Test Loss: 0.0341 | Test
Accuracy: 98.87%
Epoch 10/10 | Train Loss: 0.0153 | Train Accuracy: 99.51% | Test Loss: 0.0423 | Test
Accuracy: 98.69%
Final: Train Acc = 99.51% | Test Acc = 98.69%
```

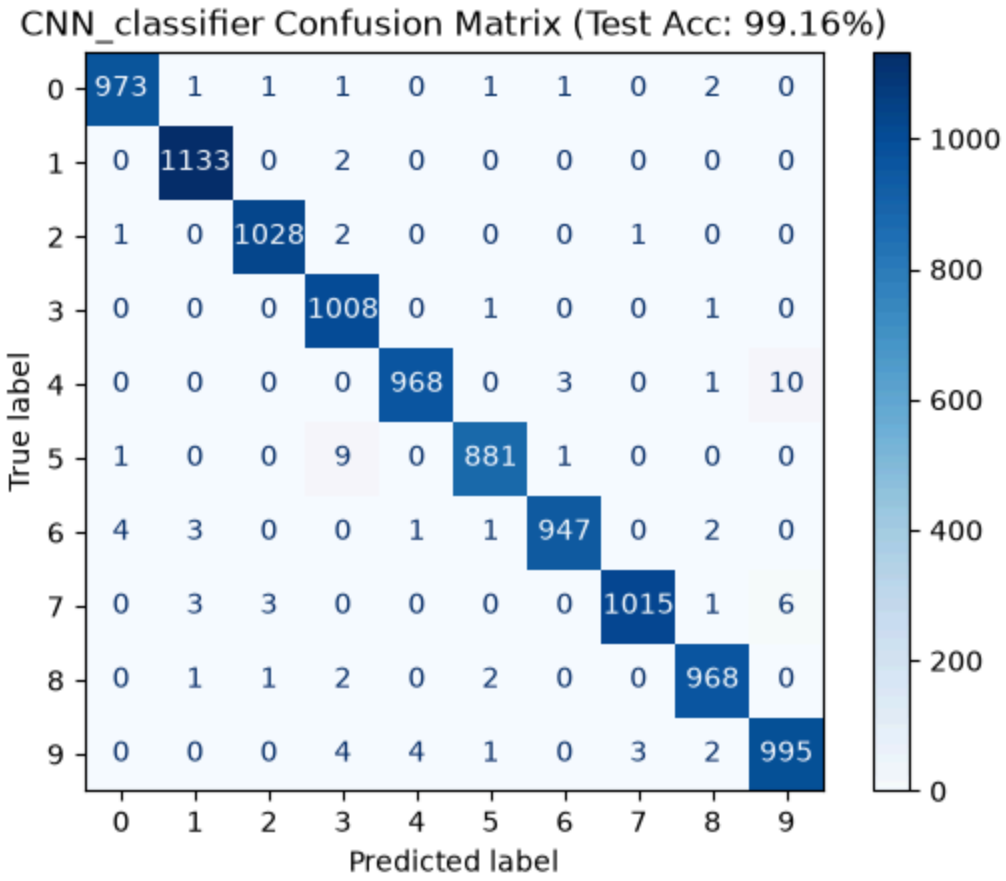







```
Training CNN_classifier on MNIST dataset for 10 epochs with lr=0.001, lr_decay=0.9,
reg=0.0001
Epoch 1/10 | Train Loss: 0.0833 | Train Accuracy: 97.30% | Test Loss: 0.0781 | Test
Accuracy: 97.32%
Epoch 2/10 | Train Loss: 0.0359 | Train Accuracy: 98.83% | Test Loss: 0.0381 | Test
Accuracy: 98.69%
Epoch 3/10 | Train Loss: 0.0245 | Train Accuracy: 99.20% | Test Loss: 0.0307 | Test
Accuracy: 98.93%
Epoch 4/10 | Train Loss: 0.0221 | Train Accuracy: 99.27% | Test Loss: 0.0315 | Test
Accuracy: 99.01%
Epoch 5/10 | Train Loss: 0.0192 | Train Accuracy: 99.40% | Test Loss: 0.0300 | Test
Accuracy: 98.98%
Epoch 6/10 | Train Loss: 0.0162 | Train Accuracy: 99.47% | Test Loss: 0.0300 | Test
Accuracy: 99.00%
Epoch 7/10 | Train Loss: 0.0105 | Train Accuracy: 99.68% | Test Loss: 0.0259 | Test
Accuracy: 99.13%
Epoch 8/10 | Train Loss: 0.0086 | Train Accuracy: 99.74% | Test Loss: 0.0253 | Test
Accuracy: 99.19%
Epoch 9/10 | Train Loss: 0.0112 | Train Accuracy: 99.63% | Test Loss: 0.0277 | Test
Accuracy: 99.13%
Epoch 10/10 | Train Loss: 0.0068 | Train Accuracy: 99.79% | Test Loss: 0.0259 | Test
Accuracy: 99.16%
Final: Train Acc = 99.79% | Test Acc = 99.16%
```





Results (parts a, b and c)

The cell above prints the log for each epoch, plots the curves and shows the confusion matrix of each model (LeNet-5 first, then the modern CNN).

Model	Parameters	Train acc.	Test acc.	Test loss	Test errors
LeNet-5	≈ 61.7 k	99.51 %	**98.69 %**	0.0423	131
Modern CNN	≈ 105.9 k	99.79 %	**99.16 %**	0.0259	84

I report the final epoch values. Picking the best epoch on the test set would be a bit unfair since there is no separate validation set.

Discussion

Learning curves

- Both models learn fast and are above 97 % test accuracy after the first epoch.
- The modern CNN reaches a lower test loss and stays lower.
- LeNet-5 shows a little over-fitting. Its training loss keeps going down, but its test loss stops improving after about epoch 5.

Confusion matrices

- Both are strongly diagonal. Most errors are between digits that look similar.
- LeNet-5 (131 errors) mostly confuses 9→7, 7→2, 4→9 and 8→2.
- The modern CNN (84 errors) reduces most of these, but 4→9 and 5→3 still remain.

Why the modern CNN is better

- Batch norm makes training more stable.
- ReLU and max-pooling avoid the saturation problem of tanh.
- Dropout and L2 reduce over-fitting.

Note: the difference is about 0.5 percentage points from a single run with one seed, and the CNN also has more parameters. So it is a small difference and I didn't test it over several runs.

Conclusion. The modern CNN got 99.16 % test accuracy compared to 98.69 % for LeNet-5. LeNet-5 is already quite good since MNIST is an easy dataset.

Question 3 — Transfer Learning with ResNet18 (Ants vs Bees)

Objective

Using the PyTorch transfer learning tutorial, take ResNet18 pre-trained on ImageNet-1K and classify the hymenoptera dataset (ants and bees) in two ways:

- (a) Fine-tuning: train all layers starting from the pre-trained weights.
- (b) Feature extraction: freeze the network and train only a new final layer.

In both cases the 1000-class output layer is replaced with a new `Linear(512, 2)` layer.

Setup

The dataset is small (about 244 training and 153 validation images), so augmentation and a pre-trained model help a lot. The validation set is used as the test set here.

- Training transforms: random resized crop 224, random horizontal flip, ImageNet normalisation
- Validation transforms: resize 256, centre crop 224, normalisation
- SGD with learning rate 0.001 and momentum 0.9, `StepLR` (× 0.1 every 7 epochs)
- Cross-entropy loss, 25 epochs, batch size 4
- The weights with the best validation accuracy are restored at the end

Libraries

In [3]:
```
# License: BSD
# Author: Sasank Chilamkurthy

import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim import lr_scheduler
import torch.backends.cudnn as cudnn
import numpy as np
import torchvision
from torchvision import datasets, models, transforms
import matplotlib.pyplot as plt
import time
import os
from PIL import Image
from tempfile import TemporaryDirectory

# Let cuDNN auto-tune convolution algorithms (faster for fixed input sizes)
cudnn.benchmark = True
# Interactive plotting mode (used by the tutorial's imshow / pause)
plt.ion()   # interactive mode
```

Out[3]:
```
<contextlib.ExitStack at 0x2ad3840d450>
```

Data loading and augmentation

In [4]:
```
# Data augmentation and normalization for training
# Just normalization for validation
# Training: random crop + flip (augmentation). Validation: deterministic resize + c
data_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
```

```
    ]),
    'val': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
}

# Hymenoptera dataset: one sub-folder per class (ants, bees) under train/ and val/
data_dir = 'hymenoptera_data'
image_datasets = {x: datasets.ImageFolder(os.path.join(data_dir, x),
                                          data_transforms[x])
                  for x in ['train', 'val']}
# Data loaders with a batch size of 4
dataloaders = {x: torch.utils.data.DataLoader(image_datasets[x], batch_size=4,
                                              shuffle=True, num_workers=4)
              for x in ['train', 'val']}
dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val']}
class_names = image_datasets['train'].classes

# We want to be able to train our model on an `accelerator <https://pytorch.org/doc
# such as CUDA, MPS, MTIA, or XPU. If the current accelerator is available, we will

device = torch.accelerator.current_accelerator().type if torch.accelerator.is_avail
print(f"Using {device} device")
```

```
Using cuda device
```

Sample training batch

A batch of augmented training images with their labels.

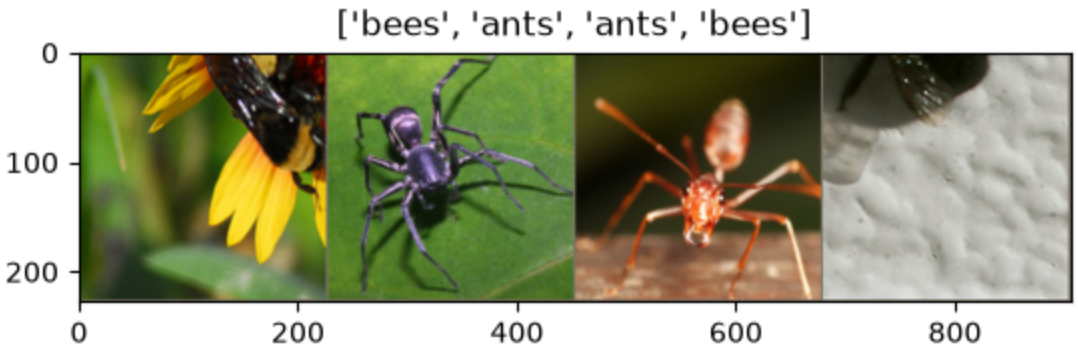
In [5]:

```
def imshow(inp, title=None):
    """Display image for Tensor."""
    inp = inp.numpy().transpose((1, 2, 0))
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    # Undo the ImageNet normalisation so that the image can be displayed
    inp = std * inp + mean
    inp = np.clip(inp, 0, 1)
    plt.imshow(inp)
    if title is not None:
        plt.title(title)
    plt.pause(0.001)  # pause a bit so that plots are updated


# Get a batch of training data
inputs, classes = next(iter(dataloaders['train']))

# Make a grid from batch
out = torchvision.utils.make_grid(inputs)

imshow(out, title=[class_names[x] for x in classes])
```



Training and visualisation functions

`train_model` runs a training phase and a validation phase in every epoch and returns the model with the best validation accuracy. `visualize_model` shows predictions on some validation images.

In [7]:
```
# Training loop: alternates a training and a validation phase each epoch and keeps
def train_model(model, criterion, optimizer, scheduler, num_epochs=25):
    since = time.time()

    # Create a temporary directory to save training checkpoints
    # The best weights are checkpointed in a temporary folder
    with TemporaryDirectory() as tempdir:
        best_model_params_path = os.path.join(tempdir, 'best_model_params.pt')

        torch.save(model.state_dict(), best_model_params_path)
        best_acc = 0.0

        for epoch in range(num_epochs):
            print(f'Epoch {epoch}/{num_epochs - 1}')
            print('-' * 10)

            # Each epoch has a training and validation phase
            for phase in ['train', 'val']:
                if phase == 'train':
                    model.train()  # Set model to training mode
                else:
                    model.eval()   # Set model to evaluate mode

                running_loss = 0.0
                running_corrects = 0

                # Iterate over data.
                for inputs, labels in dataloaders[phase]:
                    inputs = inputs.to(device)
                    labels = labels.to(device)

                    # zero the parameter gradients
                    optimizer.zero_grad()

                    # forward
                    # track history if only in train
```

```
                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                    # backward + optimize only if in training phase
                    if phase == 'train':
                        loss.backward()
                        optimizer.step()

                # statistics
                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data)
            if phase == 'train':
                # Update the learning rate once per epoch (StepLR)
                scheduler.step()

            epoch_loss = running_loss / dataset_sizes[phase]
            epoch_acc = running_corrects.double() / dataset_sizes[phase]

            print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')

            # deep copy the model
            if phase == 'val' and epoch_acc > best_acc:
                best_acc = epoch_acc
                torch.save(model.state_dict(), best_model_params_path)

        print()

    time_elapsed = time.time() - since
    print(f'Training complete in {time_elapsed // 60:.0f}m {time_elapsed % 60:.
    print(f'Best val Acc: {best_acc:4f}')

    # load best model weights
    model.load_state_dict(torch.load(best_model_params_path, weights_only=True)
return model

# Shows a few validation images together with the model's predictions
def visualize_model(model, num_images=6):
    was_training = model.training
    model.eval()
    images_so_far = 0
    fig = plt.figure()

    with torch.no_grad():
        for i, (inputs, labels) in enumerate(dataloaders['val']):
            inputs = inputs.to(device)
            labels = labels.to(device)

            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)

            for j in range(inputs.size()[0]):
                images_so_far += 1
                ax = plt.subplot(num_images//2, 2, images_so_far)
                ax.axis('off')
```

```
                ax.set_title(f'predicted: {class_names[preds[j]]}')
                imshow(inputs.cpu().data[j])

                if images_so_far == num_images:
                    model.train(mode=was_training)
                    return
        model.train(mode=was_training)
```

Part (a) — Fine-tuning

The model is loaded with the ImageNet-1K weights, the final layer is replaced, and all parameters (about 11.18 M) are trained.

In [8]:
```
# Part (a), fine-tuning: load ResNet18 with ImageNet-1K weights and train all layer
model_ft = models.resnet18(weights='IMAGENET1K_V1')
num_ftrs = model_ft.fc.in_features
# Here the size of each output sample is set to 2.
# Alternatively, it can be generalized to ``nn.Linear(num_ftrs, len(class_names))``
model_ft.fc = nn.Linear(num_ftrs, 2)

model_ft = model_ft.to(device)

criterion = nn.CrossEntropyLoss()

# Observe that all parameters are being optimized
optimizer_ft = optim.SGD(model_ft.parameters(), lr=0.001, momentum=0.9)

# Decay LR by a factor of 0.1 every 7 epochs
exp_lr_scheduler = lr_scheduler.StepLR(optimizer_ft, step_size=7, gamma=0.1)
```

```
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to C:\Users
\CHAMA COMPUTERS/.cache\torch\hub\checkpoints\resnet18-f37072fd.pth
100%|██████████| 44.7M/44.7M [00:34<00:00, 1.37MB/s]
```

In [9]:
```
# Train for 25 epochs; the weights with the best validation accuracy are restored a
model_ft = train_model(model_ft, criterion, optimizer_ft, exp_lr_scheduler,
                       num_epochs=25)
```

```
Epoch 0/24
----------
train Loss: 0.7197 Acc: 0.6639
val Loss: 0.6045 Acc: 0.8366

Epoch 1/24
----------
train Loss: 0.4851 Acc: 0.8156
val Loss: 0.3333 Acc: 0.8889

Epoch 2/24
----------
train Loss: 0.4099 Acc: 0.8361
val Loss: 0.3332 Acc: 0.8758

Epoch 3/24
----------
train Loss: 0.5382 Acc: 0.7541
val Loss: 0.4448 Acc: 0.8366

Epoch 4/24
----------
train Loss: 0.5388 Acc: 0.8115
val Loss: 0.3489 Acc: 0.8889

Epoch 5/24
----------
train Loss: 0.5203 Acc: 0.7787
val Loss: 0.5338 Acc: 0.7908

Epoch 6/24
----------
train Loss: 0.4701 Acc: 0.8689
val Loss: 0.2545 Acc: 0.9085

Epoch 7/24
----------
train Loss: 0.3849 Acc: 0.8361
val Loss: 0.2092 Acc: 0.9346

Epoch 8/24
----------
train Loss: 0.3537 Acc: 0.8770
val Loss: 0.2274 Acc: 0.9085

Epoch 9/24
----------
train Loss: 0.3490 Acc: 0.8607
val Loss: 0.1831 Acc: 0.9346

Epoch 10/24
----------
train Loss: 0.2809 Acc: 0.8689
val Loss: 0.1903 Acc: 0.9346

Epoch 11/24
```

```
----------
train Loss: 0.2590 Acc: 0.9098
val Loss: 0.1860 Acc: 0.9477

Epoch 12/24
----------
train Loss: 0.3353 Acc: 0.8648
val Loss: 0.2017 Acc: 0.9281

Epoch 13/24
----------
train Loss: 0.2488 Acc: 0.8893
val Loss: 0.1681 Acc: 0.9412

Epoch 14/24
----------
train Loss: 0.3228 Acc: 0.8811
val Loss: 0.1759 Acc: 0.9412

Epoch 15/24
----------
train Loss: 0.2615 Acc: 0.8648
val Loss: 0.1738 Acc: 0.9412

Epoch 16/24
----------
train Loss: 0.3604 Acc: 0.8648
val Loss: 0.2347 Acc: 0.9216

Epoch 17/24
----------
train Loss: 0.2222 Acc: 0.9057
val Loss: 0.1694 Acc: 0.9412

Epoch 18/24
----------
train Loss: 0.3139 Acc: 0.8730
val Loss: 0.1646 Acc: 0.9412

Epoch 19/24
----------
train Loss: 0.2644 Acc: 0.8852
val Loss: 0.1865 Acc: 0.9412

Epoch 20/24
----------
train Loss: 0.4182 Acc: 0.8074
val Loss: 0.1897 Acc: 0.9346

Epoch 21/24
----------
train Loss: 0.2412 Acc: 0.9098
val Loss: 0.1680 Acc: 0.9477

Epoch 22/24
----------
```

```
train Loss: 0.2032 Acc: 0.9262
val Loss: 0.1803 Acc: 0.9281

Epoch 23/24
----------
train Loss: 0.2613 Acc: 0.8934
val Loss: 0.1644 Acc: 0.9412

Epoch 24/24
----------
train Loss: 0.1852 Acc: 0.9385
val Loss: 0.1640 Acc: 0.9412

Training complete in 4m 50s
Best val Acc: 0.947712
```

In [10]:
```
# Predictions of the fine-tuned model on validation images
visualize_model(model_ft)
```

predicted: ants



predicted: bees



predicted: ants



predicted: bees



predicted: bees



predicted: ants



Result (a) — fine-tuning, 25 epochs

Metric	Value
Best validation accuracy	**94.77 %**
Final epoch train accuracy / loss	93.85 % / 0.185
Final epoch validation accuracy / loss	94.12 % / 0.164
Validation accuracy after first epoch	83.66 %
Training time	4 min 50 s

The validation accuracy is unstable in the early epochs (it drops to 79.08 % at epoch 5) because all the weights are changing with a fairly large learning rate. It settles at around 92–95 % after the learning rate is reduced at epoch 7.

Part (b) — Feature extractor

All pre-trained parameters are frozen (`requires_grad = False`) and only the new final layer (1 026 parameters) is trained.

In [11]:
```
# Part (b), feature extractor: load ResNet18 with ImageNet-1K weights
model_conv = torchvision.models.resnet18(weights='IMAGENET1K_V1')
# Freeze all pre-trained layers so that they are not updated during training
for param in model_conv.parameters():
    param.requires_grad = False

# Parameters of newly constructed modules have requires_grad=True by default
num_ftrs = model_conv.fc.in_features
model_conv.fc = nn.Linear(num_ftrs, 2)
```

```
model_conv = model_conv.to(device)

criterion = nn.CrossEntropyLoss()

# Observe that only parameters of final layer are being optimized as
# opposed to before.
optimizer_conv = optim.SGD(model_conv.fc.parameters(), lr=0.001, momentum=0.9)

# Decay LR by a factor of 0.1 every 7 epochs
exp_lr_scheduler = lr_scheduler.StepLR(optimizer_conv, step_size=7, gamma=0.1)
```

In [12]:

```
# Train only the new final layer for 25 epochs; the backbone stays fixed
model_conv = train_model(model_conv, criterion, optimizer_conv,
                         exp_lr_scheduler, num_epochs=25)
```

```
Epoch 0/24
----------
train Loss: 0.6320 Acc: 0.6721
val Loss: 0.2602 Acc: 0.9020

Epoch 1/24
----------
train Loss: 0.5004 Acc: 0.7541
val Loss: 0.1878 Acc: 0.9412

Epoch 2/24
----------
train Loss: 0.6609 Acc: 0.7213
val Loss: 0.7786 Acc: 0.7451

Epoch 3/24
----------
train Loss: 0.6362 Acc: 0.7336
val Loss: 0.2485 Acc: 0.9150

Epoch 4/24
----------
train Loss: 0.3374 Acc: 0.8525
val Loss: 0.2102 Acc: 0.9281

Epoch 5/24
----------
train Loss: 0.3434 Acc: 0.8607
val Loss: 0.1743 Acc: 0.9477

Epoch 6/24
----------
train Loss: 0.3716 Acc: 0.8361
val Loss: 0.1879 Acc: 0.9477

Epoch 7/24
----------
train Loss: 0.3297 Acc: 0.8770
val Loss: 0.1845 Acc: 0.9412

Epoch 8/24
----------
train Loss: 0.3930 Acc: 0.8402
val Loss: 0.1798 Acc: 0.9477

Epoch 9/24
----------
train Loss: 0.3619 Acc: 0.8320
val Loss: 0.1843 Acc: 0.9346

Epoch 10/24
----------
train Loss: 0.2986 Acc: 0.8566
val Loss: 0.1713 Acc: 0.9477

Epoch 11/24
```

```
----------
train Loss: 0.2866 Acc: 0.8811
val Loss: 0.1966 Acc: 0.9412

Epoch 12/24
----------
train Loss: 0.3663 Acc: 0.8320
val Loss: 0.1781 Acc: 0.9477

Epoch 13/24
----------
train Loss: 0.3105 Acc: 0.8607
val Loss: 0.1868 Acc: 0.9477

Epoch 14/24
----------
train Loss: 0.3832 Acc: 0.8320
val Loss: 0.1804 Acc: 0.9477

Epoch 15/24
----------
train Loss: 0.3350 Acc: 0.8279
val Loss: 0.1776 Acc: 0.9412

Epoch 16/24
----------
train Loss: 0.3556 Acc: 0.8525
val Loss: 0.1719 Acc: 0.9477

Epoch 17/24
----------
train Loss: 0.3541 Acc: 0.8238
val Loss: 0.1820 Acc: 0.9412

Epoch 18/24
----------
train Loss: 0.3934 Acc: 0.8320
val Loss: 0.1786 Acc: 0.9477

Epoch 19/24
----------
train Loss: 0.2153 Acc: 0.9098
val Loss: 0.1918 Acc: 0.9412

Epoch 20/24
----------
train Loss: 0.3247 Acc: 0.8484
val Loss: 0.1703 Acc: 0.9412

Epoch 21/24
----------
train Loss: 0.2799 Acc: 0.8730
val Loss: 0.1866 Acc: 0.9412

Epoch 22/24
----------
```

```
train Loss: 0.4011 Acc: 0.8197
val Loss: 0.2194 Acc: 0.9346

Epoch 23/24
----------
train Loss: 0.3528 Acc: 0.8238
val Loss: 0.1912 Acc: 0.9346

Epoch 24/24
----------
train Loss: 0.3118 Acc: 0.8689
val Loss: 0.1715 Acc: 0.9412

Training complete in 4m 29s
Best val Acc: 0.947712
```

In [13]:
```
# Predictions of the feature-extractor model on validation images
visualize_model(model_conv)

plt.ioff()
plt.show()
```

predicted: bees



predicted: ants



predicted: ants



predicted: bees



predicted: bees



predicted: bees



Result (b) — feature extractor, 25 epochs

Metric	Value
Best validation accuracy	**94.77 %**
Final epoch train accuracy / loss	86.89 % / 0.312
Final epoch validation accuracy / loss	94.12 % / 0.172
Validation accuracy after first epoch	90.20 %
Training time	4 min 29 s

Prediction on a single image

`visualize_model_predictions` loads one image, applies the validation transforms and shows the predicted class. Here it is used on an image from the *bees* folder with the feature extractor model.

In [14]:
```
# Predicts the class of a single image file (same preprocessing as the validation s
def visualize_model_predictions(model,img_path):
    was_training = model.training
    model.eval()
```

```
    img = Image.open(img_path)
    img = data_transforms['val'](img)
    img = img.unsqueeze(0)
    img = img.to(device)

    with torch.no_grad():
        outputs = model(img)
        _, preds = torch.max(outputs, 1)

        ax = plt.subplot(2,2,1)
        ax.axis('off')
        ax.set_title(f'Predicted: {class_names[preds[0]]}')
        imshow(img.cpu().data[0])

        model.train(mode=was_training)
```

In [16]:
```
# Classify one validation image from the 'bees' folder with the feature-extractor m
visualize_model_predictions(
    model_conv,
    img_path='hymenoptera_data/val/bees/72100438_73de9f17af.jpg'
)

plt.ioff()
plt.show()
```



Comparison and Discussion

	(a) Fine-tuning	(b) Feature extractor
Trainable parameters	≈ 11.18 M	1 026
Best validation accuracy	94.77 %	94.77 %
Final validation accuracy	94.12 %	94.12 %
Final train accuracy	93.85 %	86.89 %
Validation accuracy after first epoch	83.66 %	90.20 %

	(a) Fine-tuning	(b) Feature extractor
Training time	4 min 50 s	4 min 29 s

Discussion

- Both methods give the same accuracy (94.77 % best, 94.12 % final). With only 153 validation images, I can't say that one is better.
- Pre-trained features work very well here. Training only 1 026 parameters is enough to get about 94 %, and the feature extractor reaches 90 % after the very first epoch.
- Fine-tuning fits the training data better (93.9 % vs 86.9 %), but with so few training images this did not improve the validation accuracy.
- The training accuracy of the feature extractor is lower than its validation accuracy because the training images are randomly augmented, while the validation images are only centre-cropped.
- The training times are similar because the frozen network still has to do the forward pass for every image.
- Both runs have some dips in the early epochs (79.08 % and 74.51 %), most likely because of the small batch size (4) and the high initial learning rate.

Note: the validation set was used both to pick the best epoch and to report accuracy, so the best accuracy is slightly optimistic. Each setup was run only once.

Conclusion. Both methods get about 94–95 %. For a small dataset like this, using ResNet18 as a fixed feature extractor is cheaper and more stable. Fine-tuning would be more useful with more data or when the new data is very different from ImageNet.

Summary and Conclusions

- **Q1** – Adding a 100-node sigmoid hidden layer and using cross-entropy loss (manual back-propagation, 10 epochs) improved the accuracy from 32.22 % / 32.42 % (train / test) to **48.11 % / 45.82 %**. A fully connected network is still limited because it ignores spatial structure.
- **Q2** – After 10 epochs LeNet-5 got **99.51 % / 98.69 %** (train / test) and the modern CNN got **99.79 % / 99.16 %**, with fewer test errors (84 vs 131). The remaining errors are between similar digits like 4 and 9.
- **Q3** – ResNet18 reached **94.77 %** best validation accuracy (94.12 % final) with both fine-tuning and feature extraction. Feature extraction trains only 1 026 parameters and converges faster.

Overall, a better model structure (hidden layer, convolutions, batch norm) and pre-trained features gave better results.